

Chapitre 7 : Les pointeurs

Introduction: notion d'adresse

- La mémoire centrale utilisée par les programmes est **découpée en octets**, dont chacun est identifié par un numéro séquentiel appelé **adresse**
- Par convention, une adresse est notée en **hexadécimal** et précédée par **0x**.

0x3ffd10	
0x3ffd11	
0x3ffd12	
0x3ffd13	
0x3ffd14	

....

Introduction: notion d'adresse

Qu'est-ce qu'un pointeur :

- C'est une **variable** dont la valeur est l'adresse d'une autre variable de n'importe quel type
- C'est une **variable** qui contient l'adresse d'une autre variable
- L'adresse contenue dans un pointeur est celle d'une variable qu'on appelle **variable pointée**

Introduction: notion d'adresse

Déclarer une **variable** c'est attribuer un nom (identificateur) à une zone de la mémoire centrale

Cette zone est définie par:

-**sa position**: l'adresse de son premier octet

-**sa taille**: le nombre d'octets

x3ffd10		
x3ffd11		
x3ffd12		
x3ffd13	X	18
x3ffd14		
x3ffd15	Y	3,23
x3ffd16		

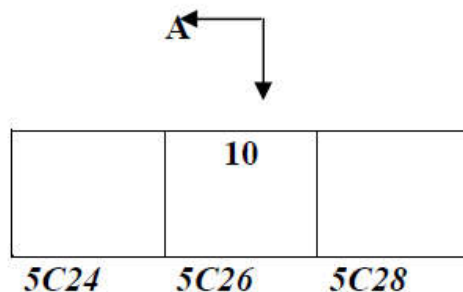
Tailles minimales garanties (norme)

Type	Taille minimale
char	1 octet (toujours 1)
short	2 octets
int	2 octets
long	4 octets
long long	8 octets
float	4 octets
double	8 octets
long double	8 octets (mais souvent 16)

Introduction: notion d'adresse

Adressage de variables:

1. **Adressage direct:** un Accès au contenu d'une variable par le nom de la variable
2. **Adressage indirect** Si nous ne voulons pas utiliser le nom d'une variable A, nous pouvons copier l'adresse de cette variable dans une variable spéciale **P** appelée Pointeur



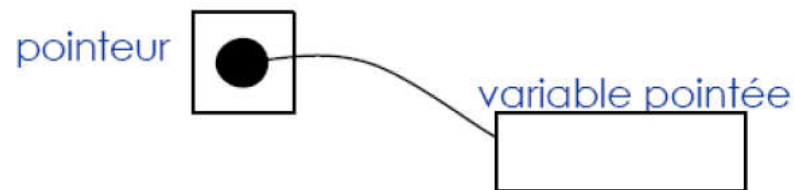
P: contient l'adresse d'une variable A

Définition

- Pour accéder à une valeur d'une variable, il suffit d'utiliser son nom
- Pour accéder à l'adresse d'une variable, on utilise un pointeur

«Un pointeur est une variable qui contient l'adresse d'une autre variable»

- Un pointeur permet d'accéder directement à la mémoire, de faire référence à toute entité, sans la nommée
- Un pointeur est associé à un **type de variable** sur lequel il peut pointer
- Par exemple, un pointeur sur entier ne peut pointer que sur des variables entières



Les variables pointeurs:

Déclaration en algorithmique:

nom_variable: pointeur sur type pointé

Exemple:

Variables p : pointeur sur entier
 x : entier

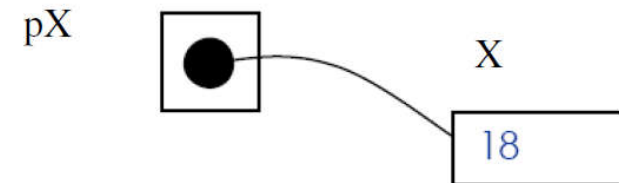
Début

lire(x)

pX ← &x

- Le pointeur pointe sur la variable dont il contient l'adresse

Représentation



0x3fffd10

0x3fffd11

0x3fffd12

0x3fffd13 X

0x3fffd14

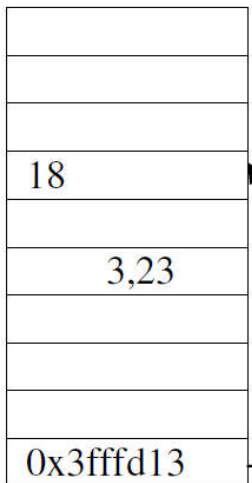
0x3fffd15 Y

0x3fffd16

0x3fffd17

0x3fffd18

0x3fffd19 pX



Les variables pointeurs:

Syntaxe générale :

nom_variable: pointeur sur type pointé

Exemple :

```
Variables
  p : pointeur sur entier
  x : entier
Début
  lire(x)
  p ← &x      // p reçoit l'adresse de x
```

Options clés:

Un pointeur contient l'adresse d'une variable.

Un pointeur est typé :

→ un pointeur sur entier ne peut pointer que vers des variables de type entier.

Le contenu de la variable pointée (la valeur stockée à l'adresse) s'obtient par : **p^**

Les variables pointeurs

Code	Code du programme
Mémoire statique	Variables globales et constantes
Pile Mémoire automatique	Piles d'appels de fonctions
Tas Mémoire dynamique Contient des trous	Allocation dynamique de mémoire

- **Mémoire statique:** zone de la mémoire où sont stockées les données qui ont la même durée de vie que le programme (variables globales)

- **Mémoire automatique:** zone de la mémoire appelée **pile** d'exécution où sont stockés les paramètres, les adresses de retour, les valeurs du retour et variables locales des fonctions empilées. Cette mémoire est gérée automatiquement par le compilateur (réservation et libération).

La mémoire est contiguë sans trous.

- **Mémoire dynamique:** zone de la mémoire appelée **tas** (top) dans laquelle le programmeur peut réserver explicitement de la place. Il devra la libérer explicitement. Cette zone est fragmentée (trouée).

Les variables pointeurs

Initialisation

Il existe **3 manières** d'initialiser une **variable de type pointeur**

1. Initialisation avec la **constante NIL**(**N**ot **I**dentified **L**ink: le pointeur ne pointe sur rien)
2. Initialiser avec **l'adresse d'une donnée existante** (adresse d'une variable ou valeur d'un autre pointeur)
3. Initialiser par **allocation dynamique de mémoire** (allouer)

NB: Pour une variable donnée, seule une des trois manières doit être utilisée

Les variables pointeurs:

1. Initialisation par NIL

- Attention à toujours initialiser un pointeur
- Un pointeur qui n'est pas initialisé s'appelle **pointeur pendant**
- Si on veut que le pointeur pointe nul part, il faut l'initialiser à NIL (NULL en C)
- C'est l'équivalent de 0 pour les entiers
- Exemple p: pointeur sur entier

$p \leftarrow \text{NIL}$ // *p pointe nul part*

Les variables pointeurs:

2. Initialisation par des données

- On affecte l'adresse d'une variable en utilisant l'opérateur «&»

Variables

pa: pointeur sur entier

a: entier

Début

$a \leftarrow 18$ (a est placé à l'adresse 0x3fffd12)

$pa \leftarrow \&a$ (pa contient l'adresse de a)

0x3fffd10	0x3fffd12	pa
0x3fffd11		
0x3fffd12	18	a
0x3fffd13		
0x3fffd14		

Les variables pointeurs

2. Initialisation par des données

Exemple 1 :

Variable

x : entier

p : pointeur sur entier

Début

x ← 33

p ← &x // p pointe sur x

écrire (p^) // la valeur 33 est affichée

p^ ← 34 // x vaut maintenant 34

Exemple 2

Variable

x, n: entier

p: pointeur sur entier

Début

n ← 20

p ← &n

x ← p^

p^ ← 30

p ← &x

n	20	x	?	p
n	20	x	?	p
n	20	x	20	p
n	30	x	20	p
n	30	x	20	p

Les variables pointeurs

3. Initialiser par allocation dynamique de mémoire (allouer)

Pour créer une variable dans le tas, on utilise l'opérateur ALLOUER suivi du type de la variable qu'on veut créer

En algorithmique:

variables

px: pointeur sur entier

Début

px ← allouer(taille(entier))

...

En C

int *px = malloc(sizeof(int));

- px est une variable de type pointeur
- allouer réserve de la mémoire dans le tas
- px contient alors l'adresse de la mémoire allouée (NIL si plus de mémoire disponible)

Les variables pointeurs

3. Initialiser par allocation dynamique de mémoire (allouer)

Libération de la mémoire dynamique

- Si on ne libère pas la mémoire, elle peut être saturée et le programme peut se planter

Libérer px

cette instruction n'efface pas le pointeur mais la variable pointée

Attention: il est interdit de

- Libérer un pointeur de valeur NIL
- Libérer une zone déjà libérée
- Accéder à une zone mémoire libérée
- Libérer de la mémoire allouée automatiquement par le compilateur (un pointeur qui pointe sur une variable de la PILE (variable nommée))

Arithmétique de pointeurs

Les seules opérations arithmétiques valides sur les pointeurs (de type entier par exemple) sont :

- **L'addition d'un entier à un pointeur.** Le résultat est un pointeur de même type que le pointeur de départ
- **La soustraction d'un entier à un pointeur.** Le résultat est un pointeur de même type que le pointeur de départ
- **La différence de deux pointeurs pointant sur des objets de même type.** Le résultat est entier

Arithmétique de pointeurs

- Si **i** est un entier et **p** un pointeur sur un objet de type X,
l'expression **p+i** désigne un pointeur sur un objet de type X dont
la valeur de l'adresse = l'adresse de p incrémentée de $i * \text{taille du type X}$
- Exemple**

```
Variable  x, y: entier
          p: pointeur sur entier

Début
  x ← 124          (x vaut 124)
  p ← allouer(taille(entier)) (p pointe sur un emplacement entier libre)
  p^ ← 1
  libérer p
  p ← &x           (p vaut l'adresse de x)
  p^ ← p^ + 1      (ce qui est pointé par p, qui est x, est incrémenté de 1)
  écrire(x)
fin
```

Exercice 1

Décrire l'évolution de chaque variable après chaque instruction : (Faire des schémas).

x, n : entier

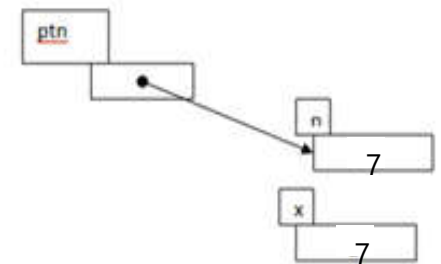
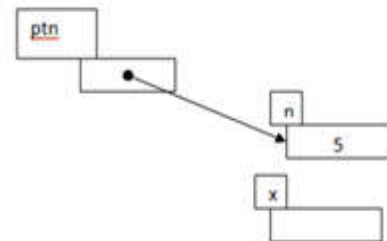
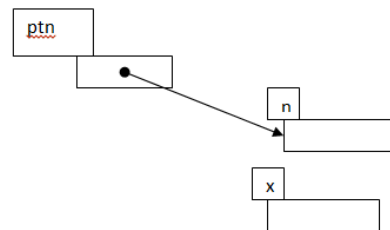
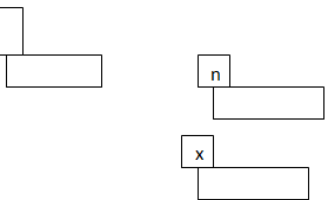
ptn : pointeur sur entier

ptn $\leftarrow$ &n

Ptn^ $\leftarrow$ 5

n $\leftarrow$ 7

x $\leftarrow$ Ptn^



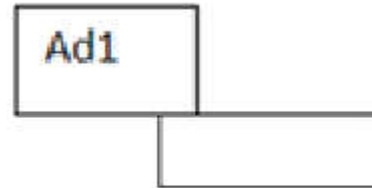
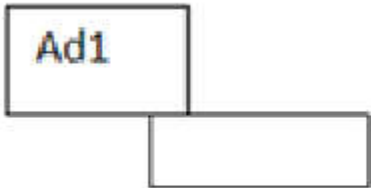
Exercice 2

Quel est le résultat des instructions suivantes : (Faire des schémas)

n : entier

Ad1 : pointeur sur entier

Ad1[←] 10



Exercice 3

Soit les trois instructions suivantes :

ad1 : pointeur sur entier

n : entier

$n \leftarrow 10$

$\text{ad1} \leftarrow \text{adresse de } n$

Que valent les expressions suivantes après les deux instructions précédentes ? (Faire des schémas).

Ad1

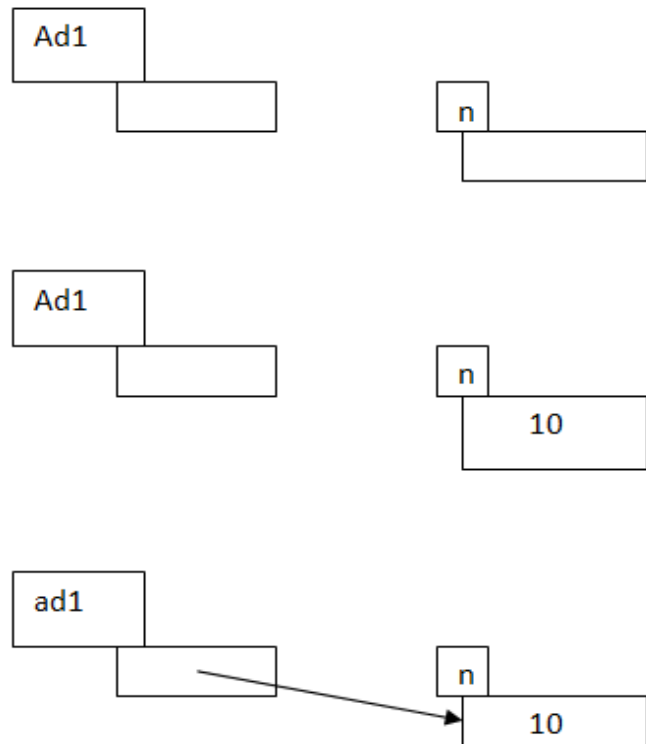
$\&\text{ad1}^{\wedge}$

$\&(\text{ad1}^{\wedge})$

n

$\&n^{\wedge}$

Exercice 3-correction



Que valent les expressions suivantes après les deux instructions précédentes ? (Faire des schémas).

`ad1 = & n`

`*(adresse de ad1) = ad1` (Contenu de l'adresse de `ad1`)

`(& ad1) = adresse n`

`n = 10`

`(&(n))^= *(ad1)=n`

Exercice 4

Ecrire une procédure dans laquelle vous :

- (a) Déclarer un entier *i* et un pointeur vers entier *p*
- (b) Initialiser l'entier *i* à une valeur arbitraire (12) et faites pointer le pointeur *p* sur *i*
- (c) Imprimer la valeur de *i* à l'écran
- (d) Modifier l'entier pointé par *p* (en utilisant *p* et non pas *i*) à une valeur arbitraire (25)
- (e) Imprimer la valeur de *i* à l'écran une dernière fois

Exercice 4-correction

Procedure Test()

(a) Déclarer un entier i et un pointeur vers entier p

```
var  
  i : entier  
  p : pointeur sur entier
```

(b) Initialiser l'entier i à une valeur arbitraire (12) et faites pointer le pointeur p sur i

```
i ← 12  
p ← &i
```

(c) Imprimer la valeur de i à l'écran

Écrire (i)

(d) Modifier l'entier pointé par p (en utilisant p et non pas i) à une valeur arbitraire (25).

```
p^ ← 25
```

(e) Imprimer la valeur de i à l'écran une dernière fois

Écrire (i)

Fin

Exercice 4 -correction

```
DÉBUT
Variables
  i : ENTIER
  p : pointeur sur entier

  i ← 12
  p ← &i

  ÉCRIRE("Valeur initiale de i : ", i)
  ÉCRIRE("Valeur initiale via p^ : ", p^)

  p^ ← 25

  ÉCRIRE("Valeur finale de i : ", i)
  ÉCRIRE("Valeur finale via p^ : ", p^)

FIN
```

Exercice 4 -correction

```
DÉBUT
Variables
  p : pointeur sur entier

  ALLOUER(p)
  p^ ← 12

  ÉCRIRE("Valeur via p^ : ", p^)

  p^ ← p^ + 13           // on change la valeur stockée

  ÉCRIRE("Nouvelle valeur via p^ : ", p^)

  LIBÉRER(p)
FIN
```

Transmission de paramètres par adresse

Nous avons vu que pour pouvoir modifier la valeur d'un paramètre dans une procédure, il fallait passer ce paramètre non pas **par valeur mais par variable**

```
Procédure Echange(Var a, b :entier)
Variable  tmp: entier
Début    tmp ← a
          a ← b
          b ← tmp
Fin
```

Le passage par adresse consiste à passer, ni la valeur ni la référence de la variable paramètre, mais l'adresse de cette variable.

Echange (**adresse de x, adresse de y**)

Transmission de paramètres par adresse

- Lors de l'appel d'une fonction, l'adresse du paramètre effectif est passé grâce à l'adresse
Echange (adresse de x, adresse de y)
- Dans l'entête de la procédure, le paramètre formel est donc un pointeur
Procédure Echange (a,b: pointeur sur entier)
- Dans le corps de la procédure, lorsqu'on veut manipuler la valeur de la variable pointée (paramètre effectif), on déréférence le paramètre formel grâce à l'opérateur $\wedge$

Procédure EchangeV2(**Var** a, b : pointeur sur entier)

Variable tmp: entier

Début tmp $\leftarrow$ a $\wedge$

a $\wedge$ $\leftarrow$ b $\wedge$

b $\wedge$ $\leftarrow$ tmp

Fin

Algorithme passage

Variable

x, y: entier

Début

x $\leftarrow$ 10

y $\leftarrow$ 20

Echange(x, y)

Ecrire (x, y)

EchangeV2(**adresse de** x, **adresse de** y)

Ecrire (x, y)

Fin

Pointeurs et tableaux

Un tableau, c'est un bloc de cases en mémoire

t : tableau[1..5] d'entiers

$t = [12, 7, 4, 9, 3]$

Le nom du tableau représente l'adresse de la 1ère case

$t \rightarrow$ adresse du premier élément

$t[1] = 12$

Un pointeur peut pointer sur un élément du tableau

p : pointeur sur entier

$p \leftarrow \&t[1]$

$p \leftarrow \&t[3]$

$p^{\wedge} = 4$

Pointeurs et tableaux

- Tout tableau, en algorithmique (et en C), est en fait un pointeur constant

Exemple:

T: tableau [1..10] d'entier T est un pointeur constant dont la valeur est l'adresse du premier élément du tableau

$T \leftarrow \text{Allouer}(N * \text{taille}(\text{entier}))$ //allouer N case pour T

T contient l'adresse de T[1]

$T^{\wedge}$ contient la valeur de T[1]

$(T+1)^{\wedge}$ contient la valeur de T[2]

T[i] correspond à $(T+i-1)^{\wedge}$

Pointeurs et tableaux

Exemples: les deux algorithmes produisent le même résultat

```
Constante N=5
P : pointeur sur entier
i: entier
Début
T[N] ← {1, 2, 3, 4, 5}
p ← adresse de T
Pour p=T à T+N-1 faire
    Écrire (p^)  
    //p ← p+1
Finpour
fin
```

```
Constante N=5
P : pointeur sur entier
i: entier
Début
T[N] ← {1, 2, 3, 4, 5}
p ← adresse de T
Pour i de 1 à N faire
    Écrire (p[i])
Finpour
fin
```

Pointeurs et Chaine de Caractères

Variables

ch : pointeur sur caractère

i : entier

Début

ch[^] ← “chaîne de caractères”

i ← 1

Tant que ch[^] <> ‘\0’ faire

 ch ← ch+1

 i ← i+1

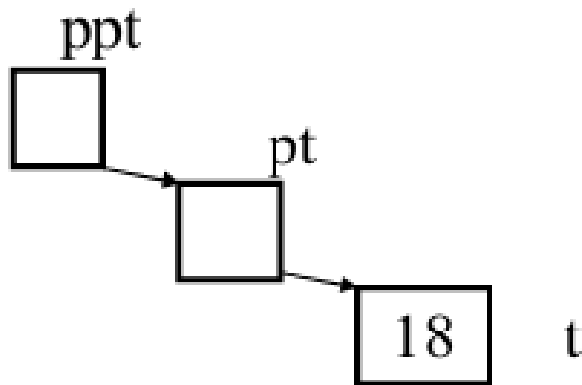
FinTQ

Écrire (“taille de la chaine est”, i-1)

fin

Double indirection

- Un pointeur pointe sur une variable: il s'agit de indirection
- Quand un pointeur **pointe sur un pointeur**, il y a double indirection



0x3ffd10	0x3ffd12	pt
0x3ffd11		
0x3ffd12	18	t
0x3ffd13		
0x3ffd14	0x3ffd10	ppt
...		

- On peut accéder à **t** par **ppt** en utilisant deux fois l'opérateur de dérèférencement
ppt^^ est équivalent à **t** et à **pt^**

Double indirection

Un tableau à 2 dimensions est un tableau de tableaux

On peut donc le voir comme un pointeur vers un pointeur

T: tableau de $[1..M, 1..N]$ d'entier

$T[1, 1]$ correspond à $T^{^^}$

$T[i, j]$ correspond à $(T[i] + j - 1)^{\wedge} = ((T + i - 1)^{\wedge} + j - 1)^{\wedge}$

Double indirection

EXEMPLE: INITIALISER UNE MATRICE

Constante M=3 N=2

Variable i,j: entier

T: matrice [1..M,1..N] d'entiers

Début

T ← allouer (M * taille(entier))

pour i de 1 à M faire

 T[i] ← allouer (N * taille(entier))

 pour j de 1 à N faire

 (T[i]+j)^ ← 0

 finpour finpour

fin

□ **LIBÉRER UNE
MATRICE**

Pour i de 1 à M faire

 libérer T[i]

Libérer T

Exercice 1

Écrire un algorithme qui lit deux tableaux A et B et leurs dimensions N et M au clavier et qui ajoute les éléments de B à la fin de A. Utiliser le formalisme pointeur à chaque fois que cela est possible.

Exercice 1-correction

Variables :

A, B : tableaux d'entiers

N, M : entiers

pA, pB : pointeurs sur entier

entier

Algorithme :

Lecture des dimensions

de N

de M

Lecture du tableau A (indices à partir de 1)

Pour i ← 1 à N **faire**

Lire A[i]

FinPour

Lecture du tableau B

Pour i ← 1 à M **faire**

Lire B[i]

FinPour

// Initialisation des pointeurs

pA ← &A[1] // pointe sur le premier élément de A

pB ← &B[1] // pointe sur le premier élément de B

// Avancer pA jusqu'à la fin logique du tableau A

Pour i ← 1 à N-1 **faire**

pA ← pA + 1 // déplacement d'un élément

FinPour

// Copie des éléments de B à la fin de A

Pour i ← 1 à M **faire**

pA ← pA + 1 // avancer pour ajouter à la fin

*pA ← *pB // copier la valeur pointée par pB dans A

pB ← pB + 1 // passer à l'élément suivant de B

FinPour

// Mise à jour de la nouvelle taille de A

N ← N + M

// Affichage du tableau A final

Pour i ← 1 à N **faire**

Afficher A[i]

FinPour

Fin

Exercice 2

Écrire un algorithme dans lequel vous déclarerez un tableau statique d'entiers `tab` avec des valeurs (10 par exemple), dont certaines seront nulles. Le programme doit lire et parcourir le tableau et imprimer les index des éléments nuls du tableau (sans utiliser aucune variable de type entier

Exercice 2-correction

Variables :

tab : tableau statique d'entiers [1..10] = [5, 0, 3, 0, 7, 0, 2, 9, 0, 4]

p : pointeur sur entier

fin : pointeur sur entier

Début

p ← adresse de tab[1] // pointeur sur le premier élément

fin ← adresse de tab[10] // pointeur sur le dernier élément

Tant que p ≤ fin **faire**

Si *p = 0 **alors**

 // Afficher l'indice de l'élément nul

 Afficher "Indice nul : ", (p - adresse de tab[1] + 1)

FinSi

 p ← p + 1 // avancer le pointeur

FinTantQue

Fin